

Principles of Network Applications

Creating a Network Application

Applications are built at the **end systems** (hosts) rather than the core of the network.

Key Idea

Applications are built by writing programs that:

- Run on different end systems.
- Communicate over the network by sending **messages**.
- Example: A web server communicates with a web browser.

Note: Core devices (routers, switches) do not run user applications. They only forward data.

Analogy

Think of the Internet like a postal system:

- The **applications** are like people writing and reading letters.
- The **network core** (routers, switches) is like the postal workers and sorting centers.
- Writers (applications) don't need to know the inner workings of the post office — only how to send/receive letters.

Benefit: Because development is only on the end systems, new applications can be created and deployed rapidly.

Network Application Architectures

Before diving into software coding, a broad **architectural plan** is needed.

- An application's **architecture** is different from the network architecture (e.g., five-layer Internet model).
- The network architecture is fixed and provides a set of services to applications.
- The **application architecture** is designed by the developer, dictating how the application is structured across end systems.

- Two predominant paradigms:
 - **Client-Server Architecture**
 - **Peer-to-Peer (P2P) Architecture**

Key Distinction

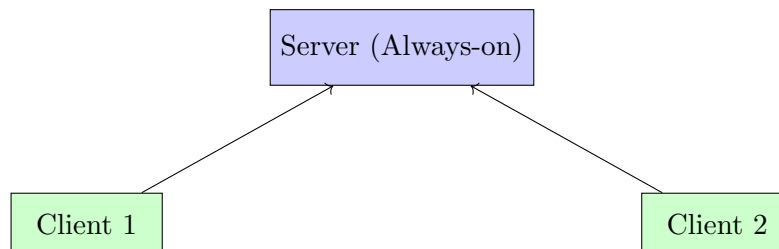
Network architecture = fixed services provided by the Internet.

Application architecture = developer's design choice on how applications are structured.

Architectural Paradigms

Client-Server Paradigm

- **Server:**
 - Always-on host with a fixed, well-known IP address.
 - Services requests from many clients.
 - Often implemented as powerful virtual servers in data centers.
- **Clients:**
 - Initiate communication with servers.
 - Intermittently connected with dynamic IP addresses.
 - Do not directly communicate with each other.
- **Examples:** Web (HTTP), FTP, Telnet, email (SMTP).



Real-World Example

Popular Internet services (Google, Amazon, Facebook, YouTube, Gmail) run on massive data centers.

- Data centers may house hundreds of thousands of servers.
- They incur costs: power, maintenance, interconnection, and bandwidth.
- Example: Google has 19 data centers worldwide handling search, Gmail, and YouTube.

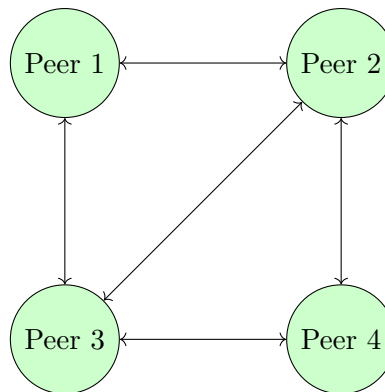
Analogy

Imagine a **restaurant**:

- The **server** = the chef (always available).
- The **clients** = customers (come and go).
- Customers never cook for each other; all requests go to the chef.

Peer-to-Peer (P2P) Architecture

- No dedicated always-on server.
- **Peers** directly communicate, acting as both clients and servers.
- Self-scalable: each new peer contributes resources but also consumes them.
- Peers are controlled by users, often laptops or desktops at homes, universities, or offices.
- Cost-effective: minimal dedicated server infrastructure needed.
- Challenges: **security, reliability, and performance**.
- **Example:** BitTorrent file sharing.



Analogy

Think of a **potluck dinner**:

- Everyone brings something to share.
- Each person is both a **provider** and a **consumer**.
- Unlike a restaurant (client-server), no single chef serves everyone.

Processes Communicating

Before building a network application, it is essential to understand how programs running in different end systems communicate. In operating system terminology, it is not actually the programs but **processes** (programs in execution) that communicate.

- Within the same host: processes communicate using **inter-process communication**, defined by the operating system.
- Across different hosts: processes communicate by exchanging **messages**.
- A **client process** initiates communication.
- A **server process** waits to be contacted.
- Even in P2P applications, a process may act as both client and server.

Key Idea

A sending process creates and sends messages into the network. A receiving process receives these messages and may respond. This communication occurs in the **application layer** of the protocol stack.

Analogy

Think of two people talking on the phone:

- One **initiates the call** (client).
- The other **answers** (server).
- Once connected, both can send and receive messages.

Client and Server Processes

A network application consists of pairs of processes that send messages to each other. For example:

- In the Web application, a **browser (client)** communicates with a **Web server**.
- In P2P file sharing, a peer downloading a file acts as the client, while the uploading peer acts as the server.

Definition

Client process: Initiates the communication session.

Server process: Waits to be contacted.

Observation

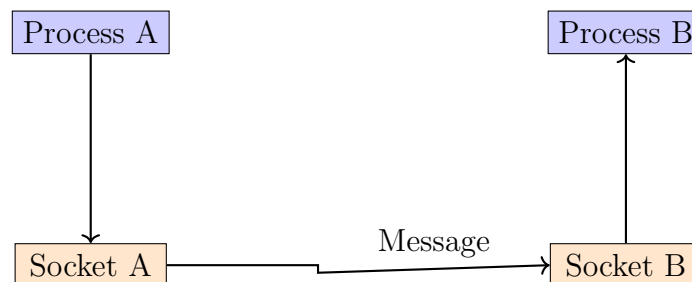
In P2P systems, a process can act as both client and server. However, in any specific communication session, one role is clearly client and the other is server.

Sockets

Most applications consist of communicating processes that exchange messages. A process sends and receives messages through a software interface called a **socket**.

Analogy

A process is like a house, and its **socket is the door**. To send a message, the process pushes it out the door (socket). The transport infrastructure carries it to the recipient's socket (door), where the receiving process retrieves it.



Real-Life Analogy

A socket is like a **mailbox**:

- You drop a letter in your mailbox (sending socket).
- The postman (network) delivers it to the recipient's mailbox (receiving socket).
- The recipient retrieves it and hands it to the right person (process).

Developer's View

A socket is the **Application Programming Interface (API)** between the application and the network. The application developer controls the application-layer side of the socket, but has limited control over the transport layer (e.g., protocol choice, buffer sizes).

Addressing Processes

To deliver messages correctly, processes must be uniquely identified.

- A host is identified by its **IP address** (32-bit in IPv4).
- An IP address alone is not enough — multiple processes may run on the same host.
- Identifier = (**IP address** + **port number**).

Common Port Numbers

- HTTP (Web server): 80
- SMTP (Mail server): 25

Bit Lengths and Port Count

- IPv4 address length: **32 bits**.
- Port number length: **16 bits**.
- Total ports: $2^{16} = 65,536$ (0–65,535).

Analogy

Think of an office building:

- The **IP address** is the building's street address.
- The **port number** is the office room number.
- Without both, mail cannot be delivered to the correct person.